1.2 Relevance of SEP forecasting to Heliophysics research

Application of ML tools to the study of SEP events will greatly benefit progress in understanding heliospheric physics. SEP events and associated drivers such as CMEs and solar flares are important dynamical solar phenomena. The study of SEPs seen in the near-Earth environment will require an understanding of the magnetic field, plasma, and turbulence structure in the solar corona and interplanetary medium. ML models will unravel the causal-effect relations among many physical variables that characterize the properties of the Sun and heliosphere. Since SEPs can cause harm to astronauts working in space and electronics on spacecraft, this investigation will carry value to space weather applications. The proposal is relevant to NASA heliophysics by addressing two of its over-arching goals stated in the decadal survey report: (Goal 2) advance our understanding of the Sun's activity, and the connections between solar variability and Earth and planetary space environments, the outer reaches of our solar system, and the interstellar medium and (Goal 3) develop the knowledge and capability to detect and predict extreme conditions in space to protect life and society and to safeguard human and robotic explorers beyond Earth.

2 Methodology in Tool Development

We propose an extension to the machine learning (ML) library Tensorflow/Keras that incorporates techniques for handling imbalanced data. Our proposed extension covers three stages of the overall machine-learning workflow: preprocessing, learning & prediction, and evaluation & analysis. As illustrated in Table 1, we plan to provide six components for the three stages. During the preprocessing stage, the extension provides stratified sampling for data partitioning. During the learning and prediction stage, the tool includes augmentation to the training distribution, decoupling representation learning and model learning, and explanation of predictions. During the evaluation and analysis stage, the extension provides evaluation metrics and visualization of learned representations.

The six components help the users handle imbalanced data and optimize their models with less effort. Our proposed Tensorflow/Keras extension with six components are based on techniques that are commonly used and/or highly cited according to Google Scholar (the number of citations is stated later for each technique); hence we consider them as TRL 5. Our extension facilitates the techniques to mature to TRL 6.

Table 1. Components of the proposed extension in three stages of machine learning workflow.

Preprocessing	Learning & Prediction	Evaluation & Analysis
Stratified sampling for data partitioning (Sec. 2.1)	Augmenting training distribution (Sec 2.2) Decoupling representation learning from model learning (Sec 2.3) Explanation of predictions (Sec 2.4)	Evaluation metrics (Sec 2.5) Visualization of learned representations (Sec. 2.6)

2.1 Stratified sampling for partitioning the training and test sets

To train and evaluate ML models, we generally partition the dataset into a larger training set and a smaller test set. The training set is for building the model, while the test set is for evaluating the model. That is, the learned model is evaluated on data that are not seen during training (or out of sample). With imbalanced data, partitioning using random sampling might not yield any minority samples in the smaller test set (or in the larger training set). Hence, we use stratified sampling to ensure that the training and test sets have the same distribution of samples. Furthermore, if the samples have associated values (such as intensity), we can perform stratified sampling to maintain the distribution of values across the training and test sets.

2.2 Augmenting training distribution via oversampling anomalous events

ML algorithms generally treat each sample of equal importance in their objective/loss functions. Hence, with imbalanced data, the minority samples collectively are less important than the majority samples. However, we are particularly interested in predicting the minority/anomalous samples. Hence, we generally increase the importance of minority samples by augmenting the training distribution via oversampling of the minority/anomalous samples (and/or under-sampling majority/frequent samples). That is, minority samples collectively have a higher importance to the ML algorithm. One way to achieve oversampling minority samples is to replicate the minority samples and train on the additional replicated samples. However, this incurs additional computational overhead.

Oversampling can also be accomplished by increasing the weights of minority samples in the objective/loss function. This way, the minority samples are not replicated and computation for training is more efficient. However, in training neural networks, we generally use a minibatch for each update of the network to reduce the issue of local minima. A minibatch contains a small subset of the training set. Since the minority samples are few, each minibatch might have one minority sample or none. When a minibatch does not have a minority sample, the network update is based on majority samples only. When a minibatch has one minority sample, which has a large weight, the minority sample could dominate the minibatch and hence the network update. That is, the updates could fluctuate between no contribution and a large contribution from the minority samples. Large fluctuations in network updates could lead to instability or no/slow convergence.

To remove the additional training computation due to replicated minority samples and large fluctuations in network updates from minibatches with weighted training samples, we plan to perform stratified sampling for a minibatch based on the weights associated with the training samples. That is, the distribution of weights for the samples in a minibatch is the same as the distribution of weights for the samples in the entire training set. Consequently, we can achieve overweighting (oversampling) the minority samples efficiently without large fluctuation in network updates. We plan to allow the user to select a range of oversampling rates and our extension can optimize the rate according to the specified objective/loss function.

2.3 Decoupling representation learning from model learning

Some of the recent advances in ML have been in learning latent features from raw/input features. Learning latent features is generally called representation/feature/embedding learning. The different layers in a neural network allow different levels of features to be learned. Recently, Kang et al. (2020) showed that separating/decoupling representation learning from model (classifier/regressor) learning can benefit ML with imbalanced data. While this approach is not common, the paper (Kang et al., 2020) has over 1.2k citations (according to Google Scholar) and

a number of variations have been proposed (Zhou et al., 2020; Wang et al., 2021). Also, results from our work (Tarsoly, 2021) indicate that the approach is beneficial.

The approach has two stages, which separate representation learning from model learning. The first stage (representation learning) focuses on learning latent features from the raw/input features with the original data distribution. That is, the latent features are learned from the natural data distribution (variation) without artificial biases. In Figure 1(a), based on the original data distribution, a typical neural network is set up to learn the latent features (z) and Classifier 1 (model). Generally, the function from input (x) to latent features (z) is called an encoder or feature extractor. The second stage (model learning) focuses on the task of prediction/detection and uses an oversampled data distribution. This has the effect of learning additional features to enhance the performance on the minority samples. In Figure 1(b), the second stage discards Classifier 1 from the first stage and learns Classifier 2 based on the oversampled distribution. Kang et al (2020) only studied classification (called cRT, classifier re-training); however, the same approach can be adapted to regression as well.

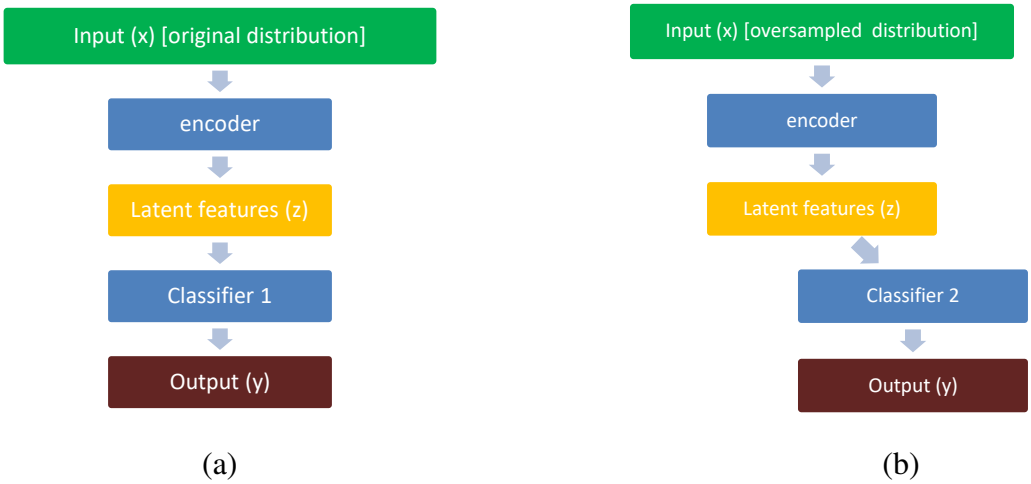

Figure 1. (a) Representation learning and (b) model learning with imbalanced data

The representation learning in stage 1 utilizes a classifier to learn the latent features. However, the goal of classification produces latent features that are *discriminative*; ie, features that can distinguish the different classes (such as SEP vs non-SEP, or normal vs anomalous). Latent features are learned that are also *generative* – features that are inherent in the data without the classes and can generate back the input samples to further enhance the representation learning in stage 1. We utilize an autoencoder, which is a common technique for generative features, to learn latent features that are generative. In Figure 2, we add a second branch to the network for representation learning. The second branch incorporates a decoder that reconstructs the input. Hence, the learned latent features (z) have characteristics of both discriminative and generative.

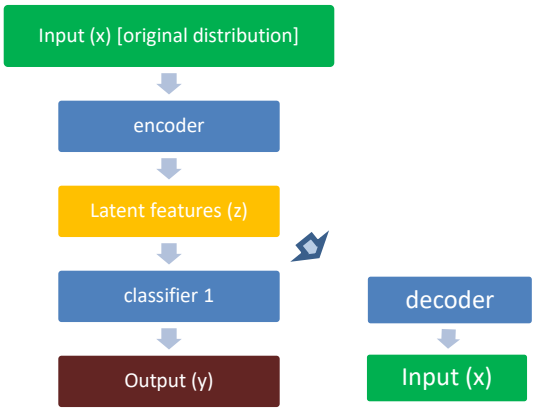

Figure 2. Representation learning with autoencoder

For our proposed extension, we plan to have decoupled representation learning and model learning as the default. The user can turn off decoupling and/or autoencoder.

2.4 Explanation of predictions

While forecasts/predictions are useful to understanding the rationale behind model predictions and trust these predictions, an explanation of the predictions and feature importance can help move beyond the assumed "black box" outputs of ML models. We plan to incorporate two commonly used methods, LIME and SHAP, to help explain a prediction made by a model for a sample. Both have libraries that we can incorporate into our extensions.

LIME (Local Interpretable Model-agnostic Explanation) proposed by Riberio et al. (2016) has over 18k citations according to Google Scholar. LIME uses a simplified binary feature space (ie, present or absent of the features) to aid interpretability. Given a data point and using the simplified features, LIME constructs a simpler model that is used to explain why the original model made the prediction. More concretely, given a data point x, LIME maps x in the original feature space into x' in the simplified features space. Samples are then drawn near x' (i.e., in the local neighborhood of x'). Using these samples in the local neighborhood, a (simpler) linear model g is learned in the simplified feature space. To be locally "faithful" to the original model f, the linear model g tries to make the same predictions as f. Features with larger weights in the linear model g provides an explanation for the prediction of x by f.

SHAP (SHapley Additive exPlanation) proposed by Lundberg and Lee (2017) has over 24k citations according to Google Scholar. SHAP is a unified approach to characterize multiple existing explanation approaches, including LIME. However, the existing approaches do not satisfy all three desirable properties: local accuracy, missingness, and consistency. The unified approach is based on Shapley values, which measure the marginal contribution of the presence vs the absence of a feature and is averaged over all subsets of features. While computation over all subsets is exponential, "Kernel SHAP" is proposed to provide more efficient approximations for model-agnostic explanations. Kernel SHAP combines Linear LIME with Shapley values. Similar to LIME, Kernel SHAP samples near the data point in the local neighborhood in the simplified feature space, but it uses a different kernel (weighting function based on distance), loss function, and model complexity metric to satisfy the three properties mentioned above.

2.5 Evaluation metrics

General evaluation metrics such as accuracy are not suitable for imbalanced data. For example, if the dataset has only 1% anomalous samples, the system has a 99% accuracy by only predicting normal. However, the predictions on all the anomalous samples are incorrect. Various evaluation metrics have been designed for imbalanced data. While F1 and AUROC (Area Under the Receiver Operating Characteristic curve) are commonly used in the ML community, the space weather community tends to use Heikde Skill Score (HSS) and True Skill Statistic (TSS) (Florios et al., 2018; Inceoglu et al., 2018).

Consider the minority/anomalous sample as Positive and the majority/normal sample as Negative, Table 2 has the confusion matrix (or contingency table) for a system.

Table 2: Confusion matrix

	Predicted Positive (PPos)	Predicted Negative (PNeg)
Actual Positive (Pos)	True Positive (TP)	False Negative (FN)
Actual Negative (Neg)	False Positive (FP)	True Negative (TN)

Based on the confusion matrix, various rates (with different names in different communities) can be defined:

- True Positive Rate (TPR) = Recall = Sensitivity = Probability of Detection (POD) = TP / Pos = TP / (TP + FN)
- False Positive Rate (FPR) = Probability of False Alarm (PFA) = FP / Neg = FP / (FP + TN)
- True Negative Rate (TNR) = Specificity = TN / Neg = TN / (FP + TN) = 1 - FPR
- Precision = TP / (PPos) = TP / (TP + FP)

Moreover, some metrics, called skill scores, incorporate a reference value that represents correct predictions (TP and TN) that can be attributed to random chance. Let S be the sample size, which is TP + TN + FP + FN. The probability of Pos is (TP+FN) / S and probability of PPos is (TP+FP) / S. The joint probability of Pos and PPos or the probability of TP is the product of these two probabilities (assuming independence between Pos and PPos). The probability of TP represents the random chance that a positive is correctly predicted. The expected value of TP is the product of probability of TP and sample size S. Similarly, the expected value of TN is estimated:

- Expected TP = Chance Hit (CH) = (TP + FN) * (TP + FP) / S
- Expected TN = (TN + FP) * (TN + FN) / S
- Expected Correct (EC) = Expected TP + Expected TN

Based on these rates and expected values, various metrics have been proposed to provide an overall metric to measure the performance of a system:

- True Skill Statistic (TSS) = TPR – FPR
- J statistic = Youden's index = Sensitivity + Specificity - 1 = TPR + TNR – 1 = TPR – (1 – TNR) = TPR – FPR = TSS
- F1 = 2 * Precision * Recall / (Precision + Recall) = 2*TP / (2*TP + FP + FN)

- Heikde Skill Score (HSS) = (TP + TN – EC) / (S – EC) = [2 * (TP*TN – FP*FN)] / [(TP+FP)(FP+TN) + (TP+FN)(FN+TN)]
- Gilbert Skill Score (GS) = (TP – CH) / (TP + FP + FN - CH)
- Critical Success Index (CSI) = Threat Score = TP / (TP + FP + FN)
- AUROC = area under the Receiver Operating Characteristic (ROC) curve [TPR vs FPR]

As shown above, TSS and J Statistic (Myer et al., 2020) are mathematically equivalent, hence only TSS is used for the following discussion. For imbalanced data, the number of Negatives (Neg) is much larger than the number of Positives (Pos). For a reasonable system, the False Positive Rate (FPR) is quite small (close to zero) due to Neg being large in the denominator. Hence, TSS is close to TPR and FP is essentially ignored. Consequently, when FP is important, TSS might not be an effective metric. F1 is the harmonic mean of Precision and Recall--FP and FN are similarly stressed with respect to TP. Hence, when FP is important, F1 is more desirable than TSS.

Heikde Skill Score (HSS) (HSS_2 in Bora & Couvidat, 2015) discounts Expected Correct (EC) due to chance from both the numerator and denominator of accuracy, which is (TP+TN) / S. Alternatively, HSS can be interpreted as the normalized difference between accuracy and the reference accuracy, which is based on the expected accuracy by chance (Hyvarinen, 2014). For a reasonable system with imbalanced data, TN is much larger than TP, FN and FP. In this case, HSS is shown to be approximately equal to F1 (Appendix in Thomas, 2022).

While HSS discounts both Expected TP and Expected TN, Gilbert Skill Score (GS) (Bora & Couvidat, 2015) does not discount Expected TN or consider TN. While GS discounts Expected TP or Chance Hit (CH), the Critical Success Index (CSI) (Crown, 2012) does not discount any expected values. CSI can be interpreted as Jaccard Index, which measures the similarity of two sets—Positives and Predicted Positives in the case of CSI. Also, F1 and CSI are quite similar; the only difference is TP is multiplied by 2 in F1, but not in CSI. That is, TP is twice as important in F1 as in CSI.

Generally, we design systems to provide a score between 0 (negative) and 1 (positive) and use 0.5 as the decision threshold to determine a sample is positive or negative. However, 0.5 might not be the most desirable threshold with respect to a specific evaluation metric. We plan to provide the user with an option to optimize the decision threshold with respect to an evaluation metric.

Unlike the previously discussed metrics, Area Under the Receiver Operating Characteristic (AUROC) curve is not based on a single decision threshold for determining positive or negative (for example, the decision threshold is 0.5). AUROC varies the decision threshold to plot the ROC curve (TPR vs. FPR), then the area under the curve is a metric that summarizes the performance over all thresholds. Although AUROC covers all thresholds, it also covers all FPRs. For predicting anomalous events, we do not desire larger FPRs. Hence, we plan to provide an option for the user to limit AUROC up to some tolerable FPR, such as 1%.

For regression tasks (output targets are continuous values such as intensity), metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Pearson Correlation are generally used and available. Skill scores incorporate a reference value that represents correct prediction due to chance. Using MSE as an example and refMSE is the reference value:

- Skill Score (SS) = 1 – (MSE / refMSE)

refMSE can be MSE of always predicting the mean (ie, expected value). That is SS measures how well the system predicts with respect to an "unskilled" system. For imbalanced data, we plan to provide an option for the user to specify a threshold to identify minority/anomalous samples and calculate the metrics separately for the minority/anomalous and majority/normal samples so that the user understands how well the system performs on the minority/anomalous samples.

We plan to provide metrics that are not in the Tensorflow library and provide aliases to metrics that are equivalent so that the users can use metric names that are more common in their community. Also, we plan to provide the confusion matrix so that additional metrics from different communities can be added.

2.6 Visualization of learned representations

Generally, the number of learned latent features is larger than 2 or 3, which makes visualizing the samples in the latent feature space difficult. To help the user visualize the samples in the latent feature space, we plan to utilize t-SNE (Van der Maaten and Hinton, 2008), which displays samples in a 2D space. The paper by Van der Maaten and Hinton (2008) has over 45k citations according to Google Scholar, and t-SNE is widely used by ML researchers in understanding the representations of samples in the latent feature space and comparing different latent feature spaces for the same problem/task. t-SNE estimates the joint probability of two samples being the nearest neighbor to each other in the higher dimensional latent feature space. Then t-SNE tries to find 2-D coordinates for the samples that yield a similar joint probability distribution as measured by Kullback-Lieber Divergence. In other words, if two samples are farther away (closer together) in the higher dimensional feature space, they are farther away (closer together) in the 2D t-SNE space. Scikit-learn provides t-SNE via sklearn.manifold.TSNE. We plan to incorporate that library into our extension. If the user selects the t-SNE option, our extension will provide a t-SNE visualization based on the second-last layer of the network (before the final output layer). Optionally, the user can select a different layer for visualization.

3 Validation in SEP Forecasting

We use three types of features: (1) features that are observations and features derived from them, (2) features that are calculated based on hand-crafted physics-based models, and (3) features that do not currently have hand-crafted physics-based models and are ML-based models trained from observations. Using these features, we plan to build models to forecast SEP targets (y), such as onset time, peak time, peak intensity, rate of intensity change, and the duration of SEP events.

3.1 Observational Features

CME Dataset. The Coordinated Data Analysis Workshop (CDAW CME) catalog maintained by the National Space Science Data Center (NSSDC), NASA Goddard Space Flight Center (GSFC) containing all the CMEs identified manually since the launch of the SOHO mission (1996-2018) can be downloaded at https://cdaw.gsfc.nasa.gov/CME_list/. The two telescope C2 and C3 (the C1 telescope was disabled in June 1998) of the Large Angle and Spectrometric Coronagraph (LASCO) monitor the emission from the region around the sun continuously. The date and time that a CME first appears in the LASCO/C2 field of view, the central position angle, the sky-plane

4 Programming Interface for the Proposed Extension

The proposed extension complements existing tools (Tensorflow/Keras) to learn from imbalanced data and it is not a standalone tool. That is, our extension alone is not sufficient to learn a model. For example, using existing tools, the users need to input/clean data, design the neural network, and produce output for their tasks. Similar to most software extensions (or API, application programming interface), our extension is encapsulated as a library/package that provides interfaces to the proposed functionalities. In the Python programming language, packages are incorporated via the keyword "import" in a program. Similar to Tensorflow/Keras, our extension is imported. Similar to Keras extending the functionalities of Tensorflow, our extension extends the functionalities of Tensorflow/Keras.

4.1 An example for incorporating our extension

On the left side of Table 3, we have an example of how a simple model can be built and evaluated in Python. In the comments, we identify 5 parts. Part 1 imports libraries/packages to extend the functionality of Python. Part 2 reads in the training set and test set for the model. Part 3 specifies the structure of the neural network for the model. Part 4 specifies the loss function, evaluation metric, and training data. Part 5 evaluates the model on the test set.

Our proposed extension is designed to be incorporated in Parts 1, 4 and 5, which are highlighted on the right side in Table 3. Also, we add Part 6 for explanation and visualization. Part 1 imports the package from our extension called `imbal` (imbalanced data). In Part 4, `imbal.compile_fit_crt()` combines the compile and fit methods to facilitate decoupling representation from classification learning (crt stands for classifier re-training as discussed in Sec 2.3). Stratified sampling (Sec 2.1) and augmenting the data distribution (Sec 2.2) are encapsulated in `imbal.compile_fit_crt()`. Also, an evaluation metric such as `hss` (Sec 2.5) is specified as a parameter to `imbal.compile_fit_crt()`. Part 5 prints HSS instead of Accuracy. Part 6 incorporates Explanation and Visualization (Sec 2.4 and Sec 2.6) via two methods: `imbal.explain_snap()` and `imbal.visualize_tsne()`.

4.2 Generalizability, Documentation, and Open-source Software

In the previous subsection, our simple example of incorporating our extension in Table 3 does not rely on SEP forecasting or Heliophysics. That is, the proposed extension can be used beyond SEP forecasting or Heliophysics. Furthermore, our extension does not impact the structure of a neural network--Part 3 in Table 3. That is, our extension can be used with different neural network models.

The software will be documented at the file level as well as the method level in the source code. A webpage will document all the methods and their parameters and return values. The user guide will include examples of how to incorporate our extension in the different parts of the program (similar to Table 3). A synthetic dataset and expected results will be included in the examples. Furthermore, tutorial videos will be provided to go through each example step by step as part of the training process.

The software will be posted on Github as Open-Source Software (OSS). Generally, users can freely reuse or adapt the software for non-commercial purposes via a GPL license. Bug fixes and new features will be incorporated by the user community and us via Github. We plan to publicize our extension via mailing lists for the heliophysics community.

Table 3. A simple example on training and evaluating a model and incorporating our extension

A simple example on training and evaluating a model	Incorporating our extension into the simple example

`# 1. importing packages`
 	
`import keras`
 	
`from keras import layers`
 	
`# 2. Reading in training set (x_train, y_train) and test set (x_test, y_test) are read`
 	
`# 3. Specifying structure of a neural network`
 	
`inputs = keras.Input( ...)`
 	
`hidden = layers.Dense( … )(inputs)`
 	
`outputs = layers.Dense(…)(hidden)`
 	
`model = keras.model( inputs=inputs, outputs=outputs, ….)`
 	
`# 4. Specifying loss function, evaluation metrics, and training data`
 	
`model.compile((loss= … , metrics=["accuracy"])`
 	
`history = model.fit( x_train, y_train, …)`
 	
`# 5. Evaluating the model on the test set`
 	
`test_scores = model.evaluate( x_test, y_test, ...)`
 	
`print("Test loss: ", test_scores[0])`
 	
`print("Test accuracy: ", test_scores[1])`
	`# 1. importing packages`
	
	`import keras`
	
	`from keras import layers`
	
	`import imbal`
	
	`# 2. Reading in training set (x_train, y_train) and test set (x_test, y_test) are read`
	
	`# 3. Specifying structure of a neural network`
	
	`inputs = keras.Input( ...)`
	
	`hidden = layers.Dense( … )(inputs)`
	
	`outputs = layers.Dense(…)(hidden)`
	
	`model = keras.model( inputs=inputs, outputs=outputs, ….)`
	
	`# 4. Specifying loss function, evaluation metrics, and training data`
	
	`history = imbal.compile_fit_crt(model, loss=..., metrics=["hss"], x_train, y_train, …)`
	
	`# 5. Evaluating the model on the test set`
	
	`test_scores = model.evaluate( x_test, y_test, ...)`
	
	`print("Test loss: ", test_scores[0])`
	
	`print("Test HSS: ", test_scores[1])`
	
	`# 6. Explanation and Visualization`
	
	`y_test_pred = model.predict(x_test,…)`
	
	`imbal.explain_shap(model, x_test, y_test_pred, ...)`
	
	`imbal.visualize_tsne(model, x_test, y_test_pred, ...)`